The Razor Pages framework exposes the `AddRazorPages()` extension method, for example, which adds all the necessary framework services to your app. Invoke the extension method on the `Services` property of `WebApplicationBuilder` in Program.cs, as shown in the following listing.

Listing 8.6 Registering the Razor Pages services with the DI container

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddRazorPages();     ❶

WebApplication app = builder.Build();
app.MapRazorPages();                  ❷
app.Run();
```

❶ The AddRazorPages extension method adds all necessary services to the IServiceCollection.

❷ Registers all the Razor Pages in your application as endpoints

It’s as simple as that. Under the hood, this call is registering multiple components with the DI container, using the same APIs you’ll see in chapter 9 for registering your own services.

NOTE Don’t worry about the Razor Pages aspect of this code; you’ll learn how Razor Pages work in part 3. The important point of listing 8.6 is to show how to register and enable various features in ASP.NET Core.

Most nontrivial libraries that you add to your application will have services that you need to add to the DI container. By convention, each library that has necessary services should expose an `Add*()` extension method that you can call on `WebApplicationBuilder.Services`.

There’s no way of knowing exactly which libraries will require you to add services to the container; it’s generally a case of checking the documentation for any libraries you use. If you forget to add them, you may find that the